Bauhaus-Universität Weimar

Model checking for Self-Replicating Robotic Systems

Student:Dmitry Babaytsev

Referees: Prof. Dr. Jan Oliver Ringert
PD Dr. Andreas Jakoby

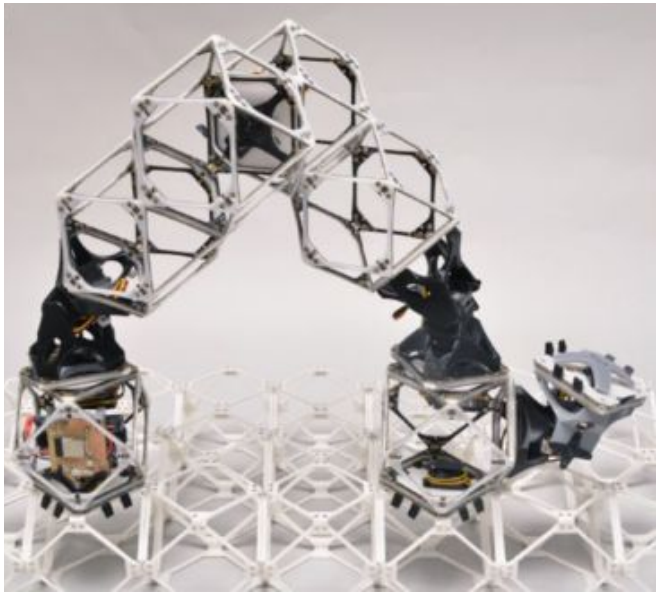
4 November 2025

Introduction

This work investigated the application of automated model checking to the domain of self-replicating robotic systems (SRRS).

The structure of the presentation:

- SRRS domain background
- Model checking background
- Research questions and the subject SRRS
- Simulation platform
- Implementation of model checking aspects to SRRS
- Threats to validity
- Conclusion



Self-replicating robotic systems

A self-replicating robotic system is a system capable of creating an identical or very similar copy of itself using available resources, without significant changes to its design or structure.

Formal description

Von Neumann:

$$(A + B + C + D) \rightarrow (A + B + C + D), (A + B + C + D)$$

A - constructor
B - copier
C - controller
D - instructions

Degree of self-replication:

$$DOSR = \frac{System\ Complexity}{Parts\ Complexity}$$

G.S. Chirikjian:

$$(R, M, E) \rightarrow (R', M', E')$$

$$(R', M', E') = \Phi(R, M, E, t)$$

R - functional system to be reproduced
M - multiset of available parts
E - environmental structures
R' , M' , E' , the replicated entities, the remaining or altered material and the possibly changed environment respectively.

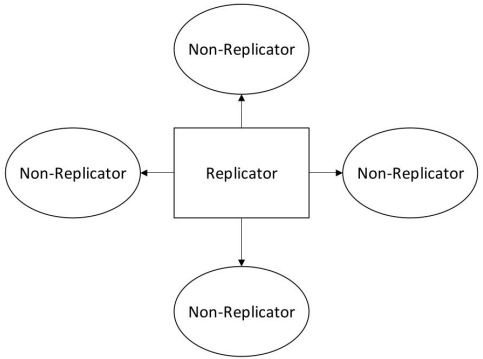
Gregory S. Chirikjian. “Entropy, Symmetry, and the Difficulty of Self-Replication”. In: arXiv preprint arXiv:2202.02938 (2022) (cit. on pp. 12, 20).

Self-replicating robotic systems

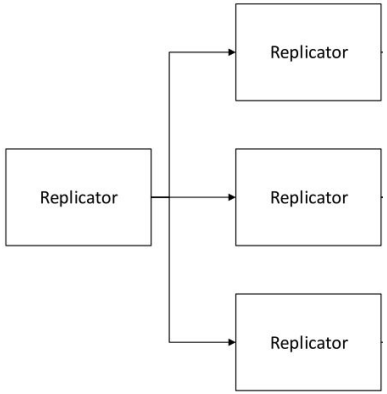
A self-replicating robotic system is a system capable of creating an identical or very similar copy of itself using available resources, without significant changes to its design or structure.

Classification of replication process

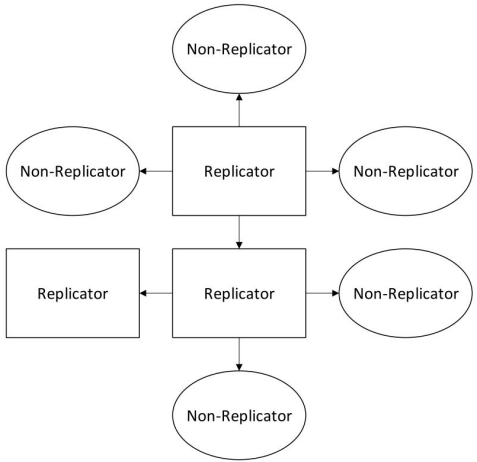
Centralized



Decentralized



Hierarchical

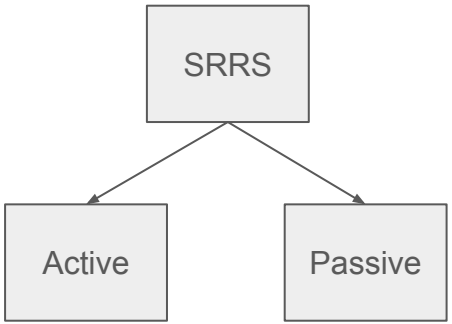


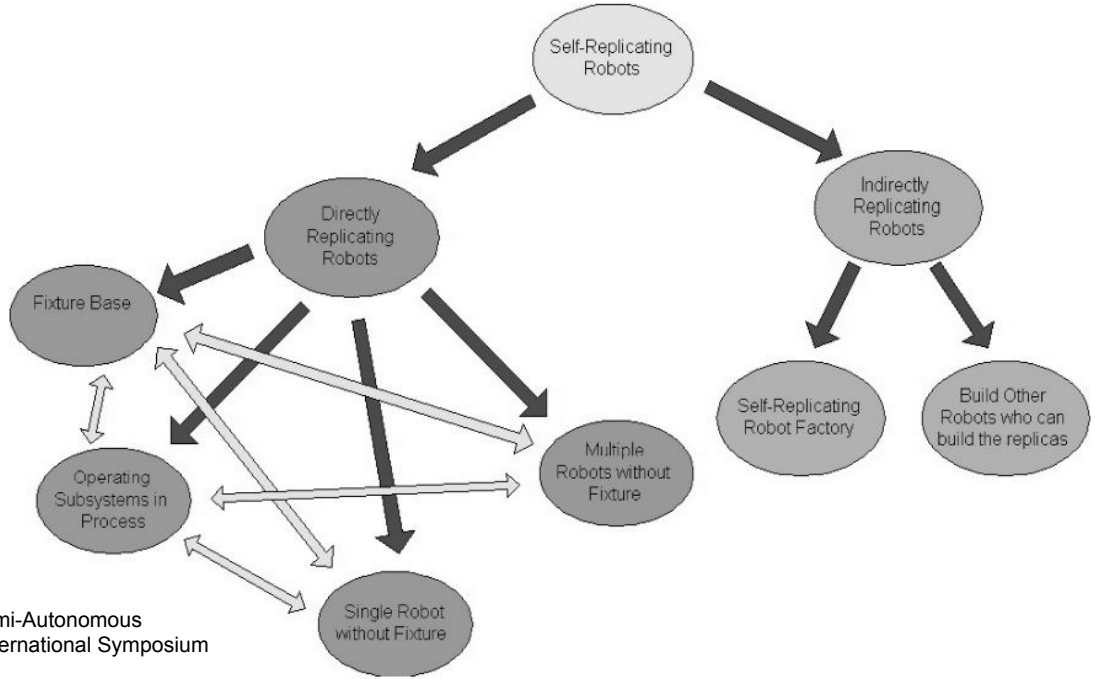
Andrew Jones and Jeremy Straub. “Simulation and Analysis of Self-Replicating Robot Decision-Making Systems”. In: Computers 10.1 (2021),

Self-replicating robotic systems

A self-replicating robotic system is a system capable of creating an identical or very similar copy of itself using available resources, without significant changes to its design or structure.

Classification of replication process

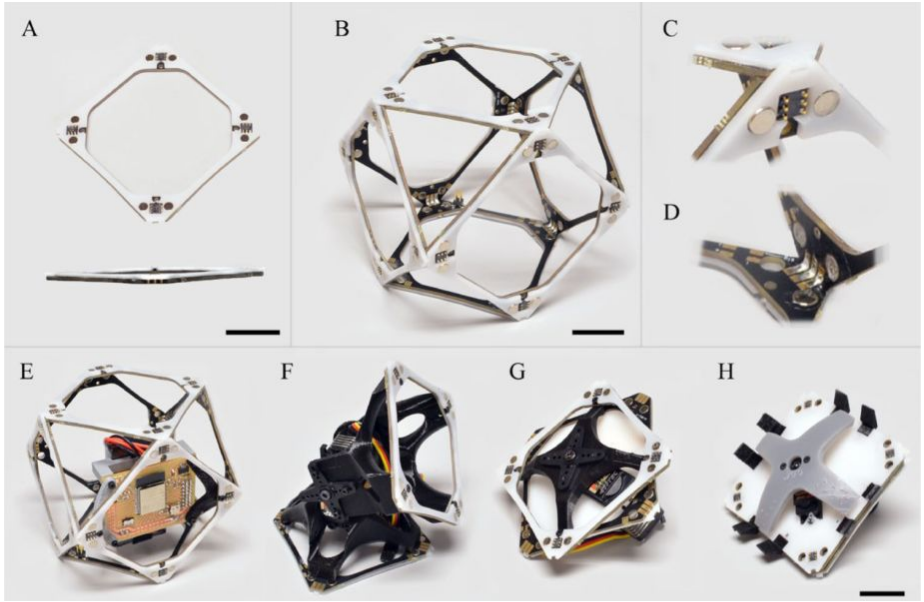




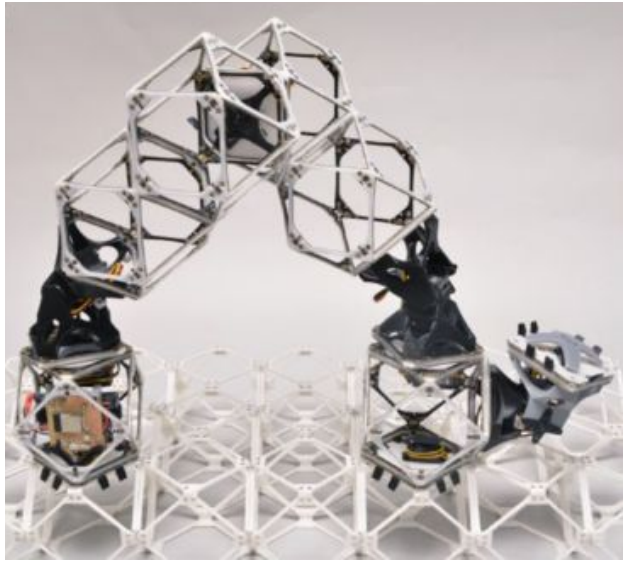
Jackrit Suthakorn, Yong T. Kwon, and Gregory S. Chirikjian. “A Semi-Autonomous Replicating Robotic System”. In: Proceedings of the 2003 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA).

Self-replicating robotic systems

Implementation of an active, kinematic, decentralised directly replicating SRRS



A Voxel face consisting of laminated PCB and acetal overlay.
B Assembled voxel
C-D Detail view of hermaphroditic magnetic and electrical connections.
E-H Different types of blocks



Assembler robot on passive lattice base.

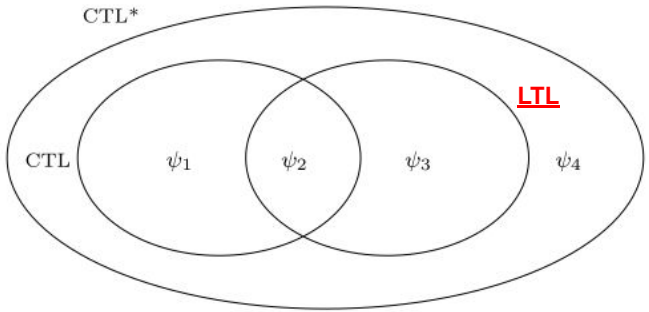
Amira Abdel-Rahman, Christopher Cameron, Benjamin Jenett, Miana Smith, and Neil Gershenfeld. “Self-replicating hierarchical modular robotic swarms”. In: Communications Engineering 1 (2022)

Model checking

Model checking is a formal verification method used to automatically determine whether a model of a system satisfies a given specification.

Models M - transition systems

Properties ϕ - formulas in temporal logic.



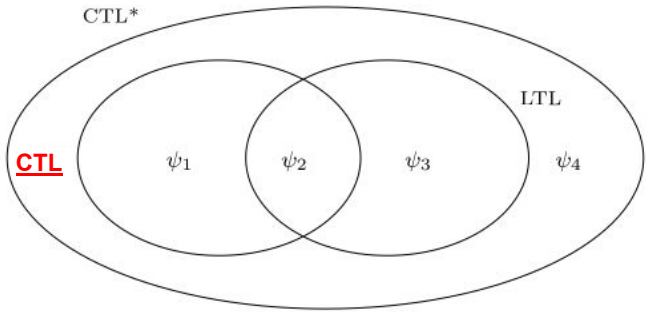
Operator	Symbol	Meaning
Next	X φ	φ holds in the **next** state
Eventually	F φ	φ will hold **sometime in the future**
Always	G φ	φ holds **at all future times**
Until	φ U ψ	φ holds **until** ψ becomes true
Release	φ R ψ	ψ must hold **until and including** φ holds

Model checking

Model checking is a formal verification method used to automatically determine whether a model of a system satisfies a given specification.

Models M - transition systems

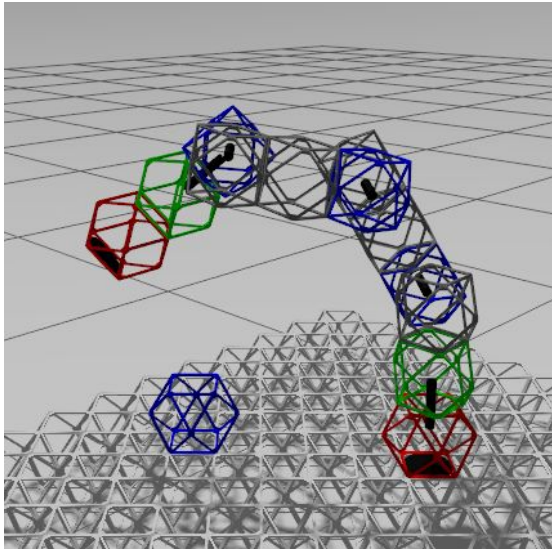
Properties ϕ - formulas in temporal logic.



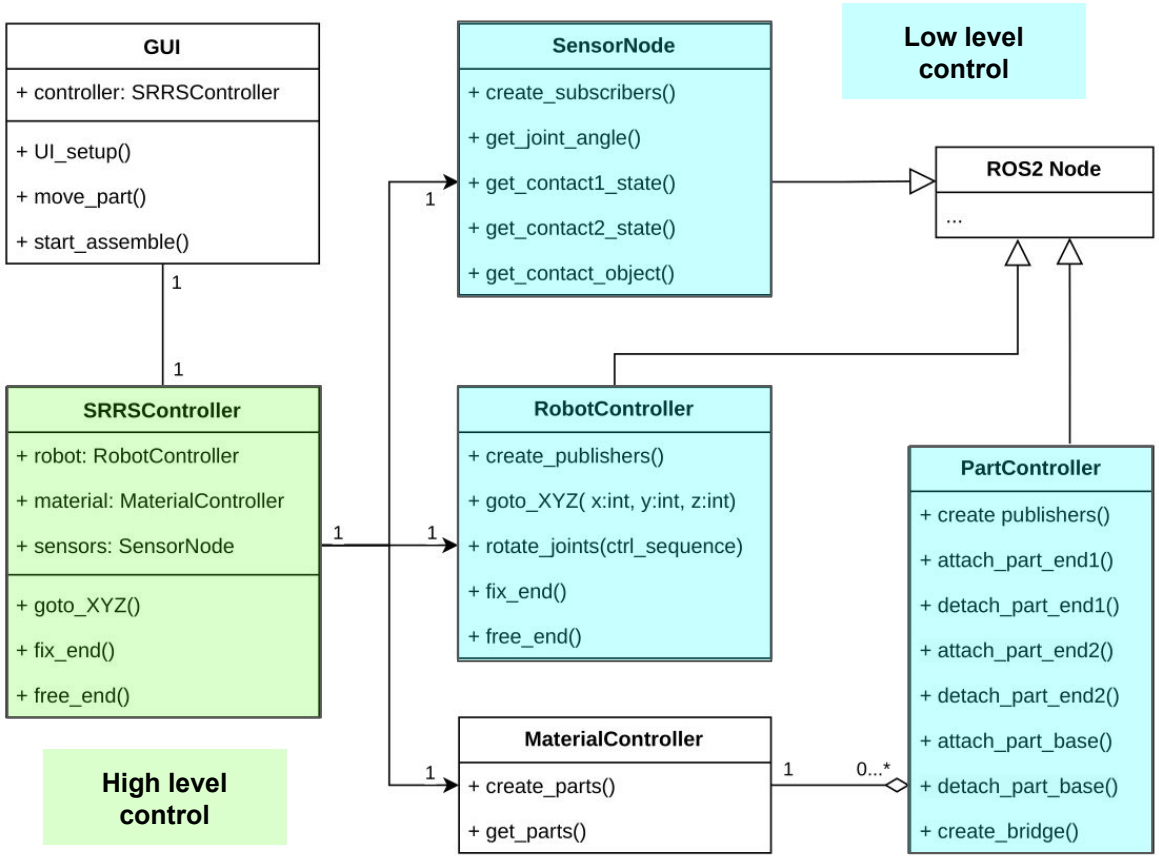
Operator	Symbol	Meaning
AX φ	A X φ	φ holds in the **next state of all paths**
EX φ	E X φ	φ holds in the **next state of some path**
AF φ	A F φ	On **all paths**, φ will eventually hold
EF φ	E F φ	There **exists a path** where φ eventually holds
AG φ	A G φ	On **all paths**, φ holds globally (always)
EG φ	E G φ	There **exists a path** where φ always holds
A[φ U ψ]	A (φ U ψ)	On **all paths**, φ holds until ψ
E[φ U ψ]	E (φ U ψ)	There **exists a path** where φ holds until ψ

Research questions

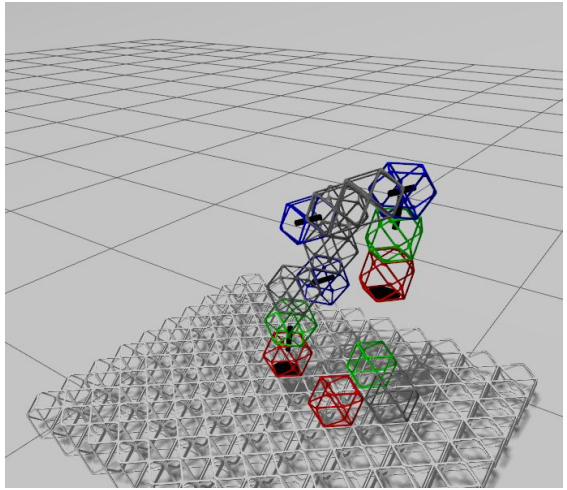
1. How can automated model checking be used to verify different properties of self-replicating robotic system, and what adaptations can be done for practical implementation?
2. What systematic methodology can translate the behaviors and structural constraints of self-replicating robots into formal models and temporal-logic specifications?
3. What limitations – computational, semantic, or modelling-related – does model checking face in domain of SRRS, and how these limitations can be mitigated?



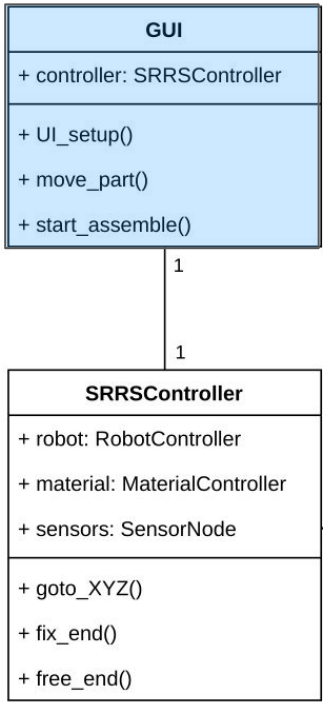
Experimental platform for simulation



ROS2 based distributed control system Gazebo

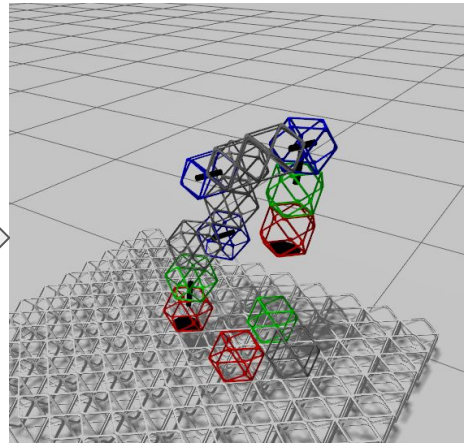


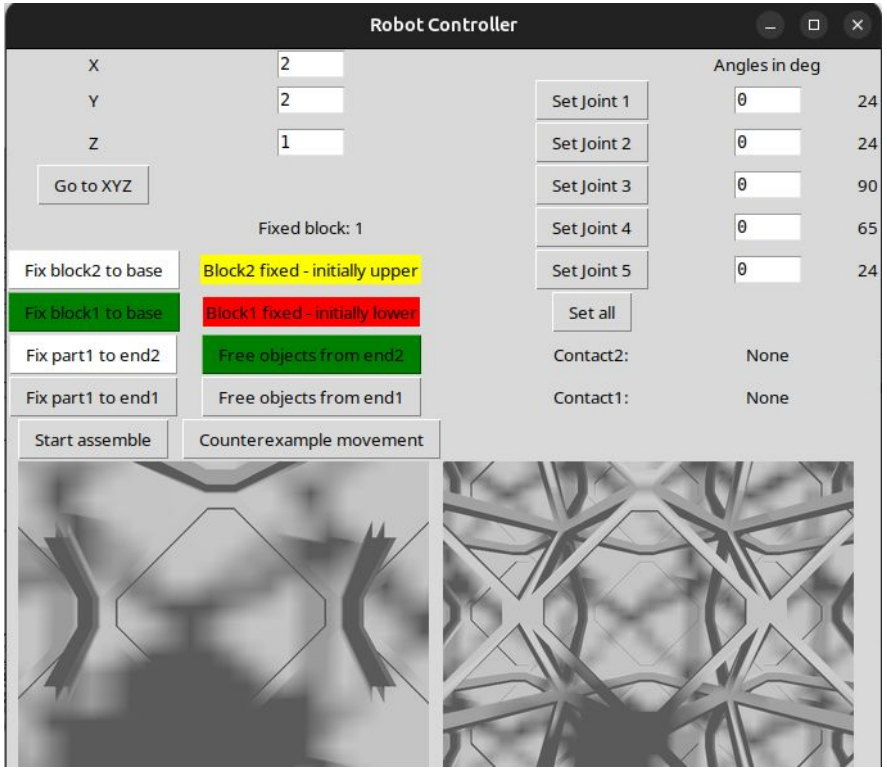
ROS2-Gazebo simulation



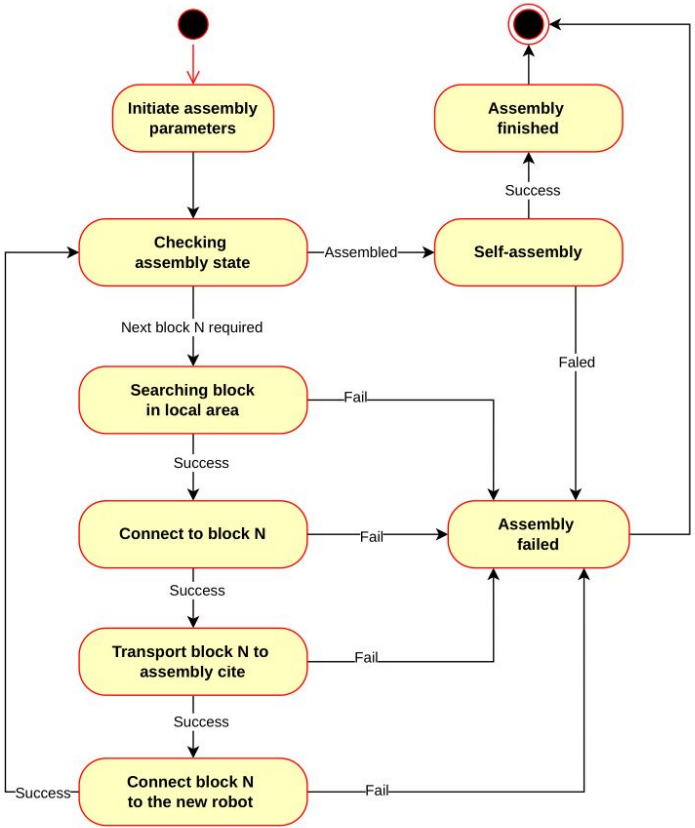
User control

Gazebo simulator

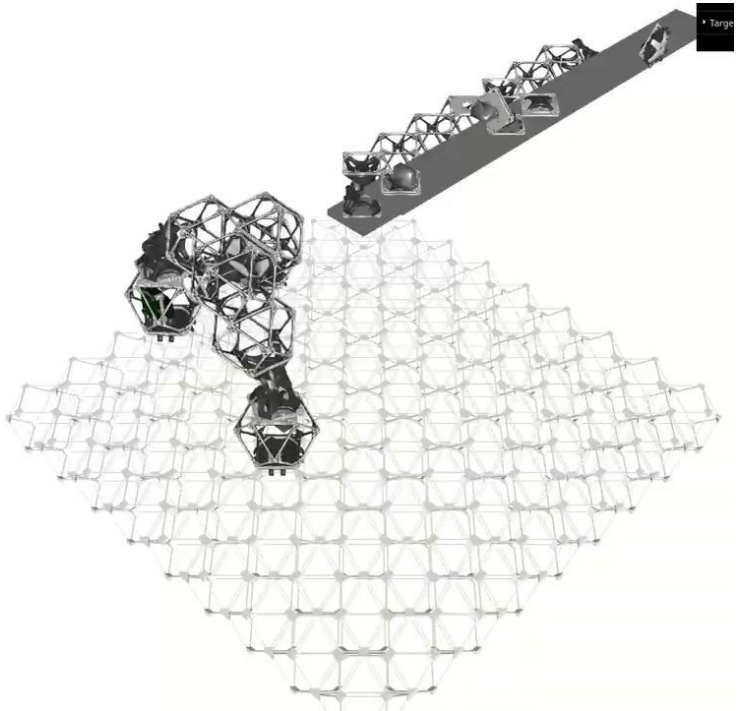




Analysis of the applicability of model checking to SRRS

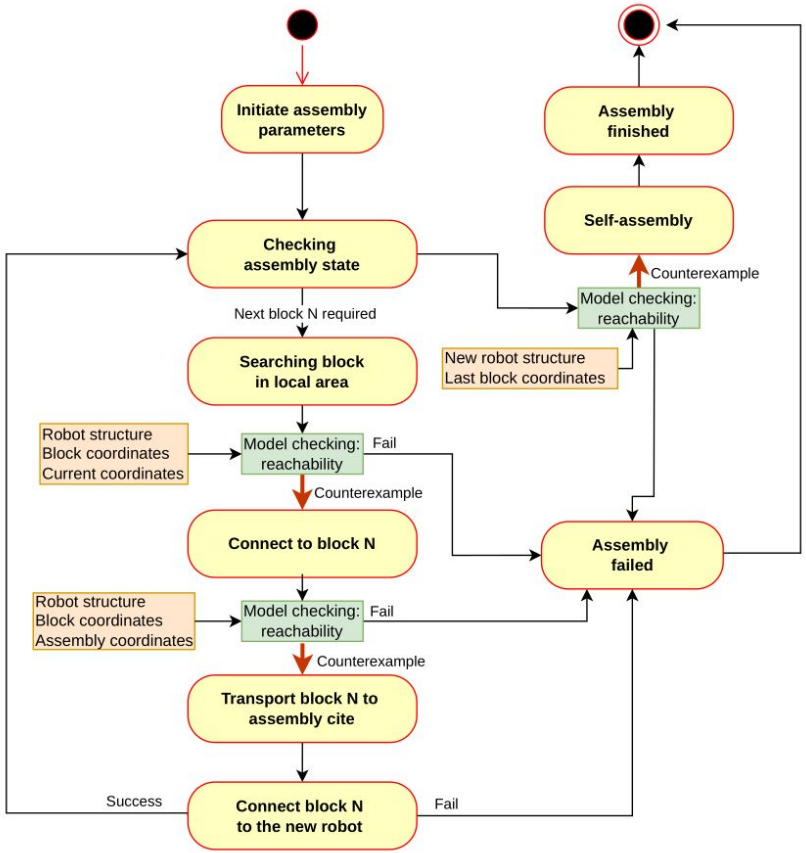


General self-replication process



Amira Abdel-Rahman, Christopher Cameron, Benjamin Jenett, Miana Smith, and Neil Gershenfeld. “Self-replicating hierarchical modular robotic swarms”. In: Communications Engineering 1 (2022)

Analysis of the applicability of model checking to SRRS

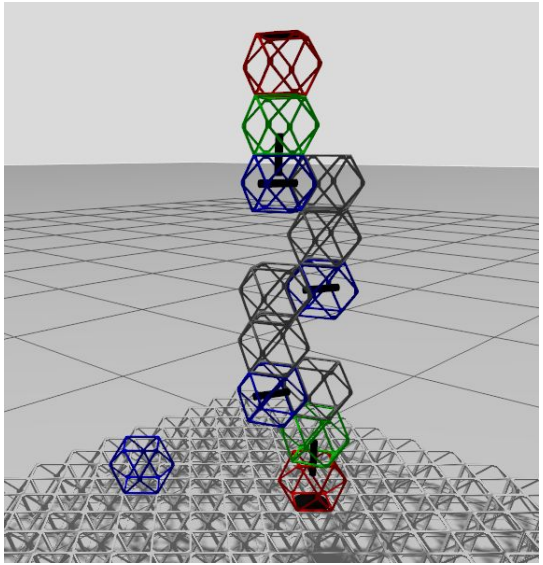


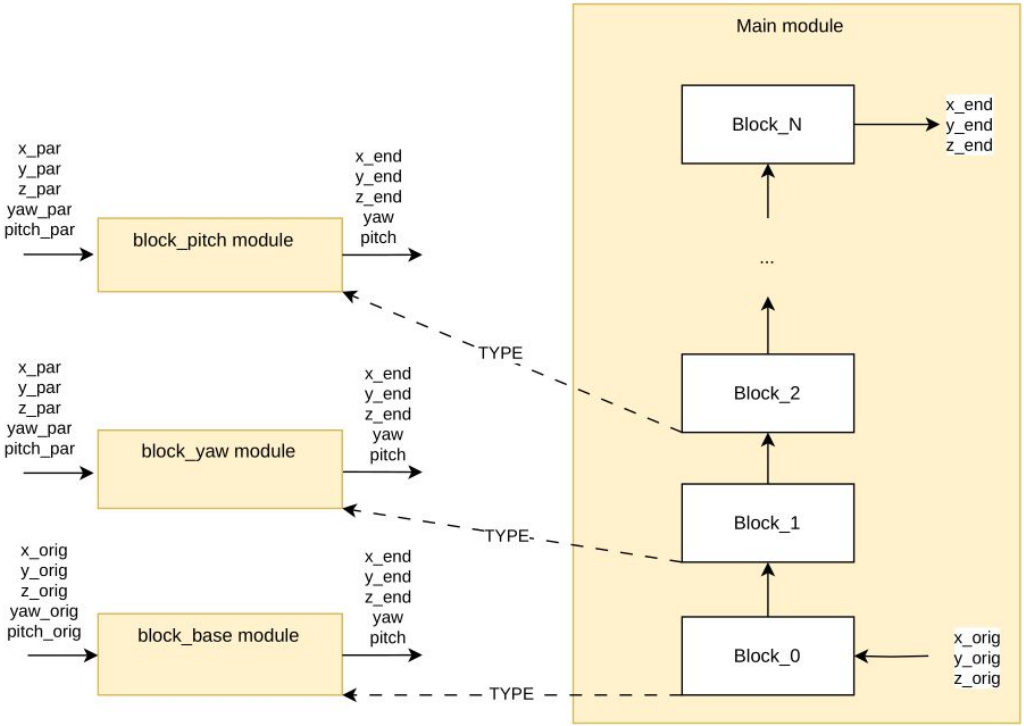
Cases:

1. Check local reachability of some point or a set of points (region) by the robot end-effector.
2. Research global reachability of some point or region by robot end-effector using Implementation of model checking in SRRS the walking form of movements.
3. Analyze set of robot configurations – check existing configuration or find relevant configurations of blocks based on counterexamples.

Usage of the counterexample

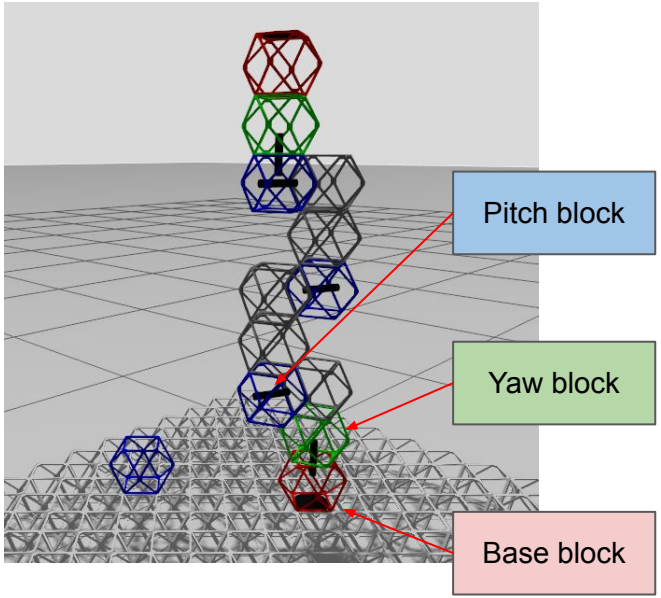
Local reachability model checking

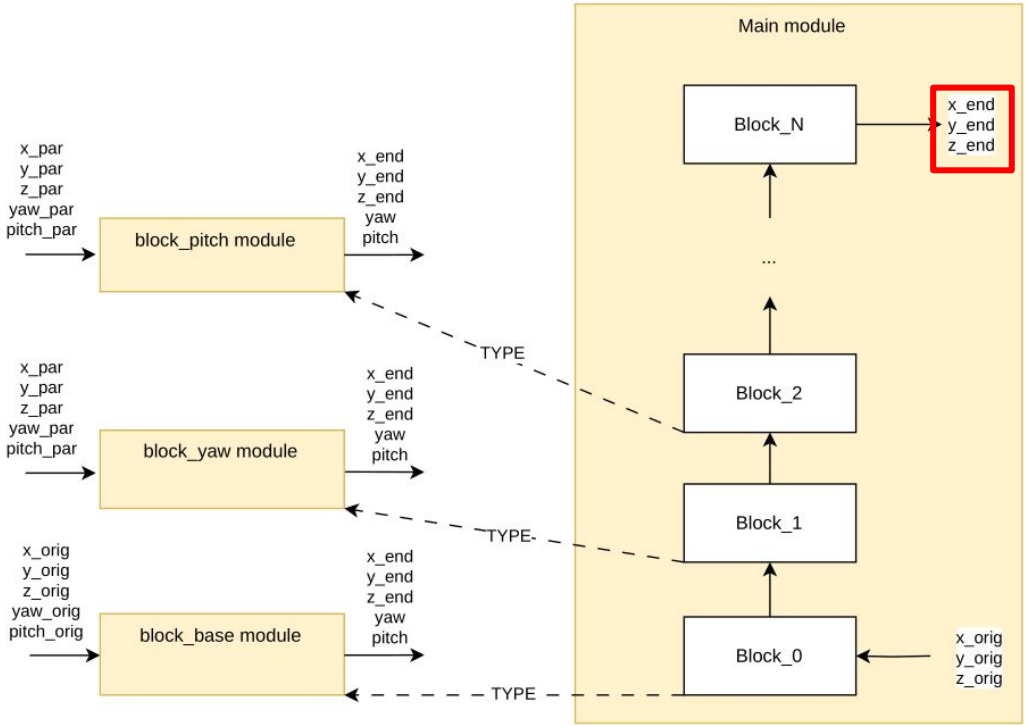




NuSMV model of a robot

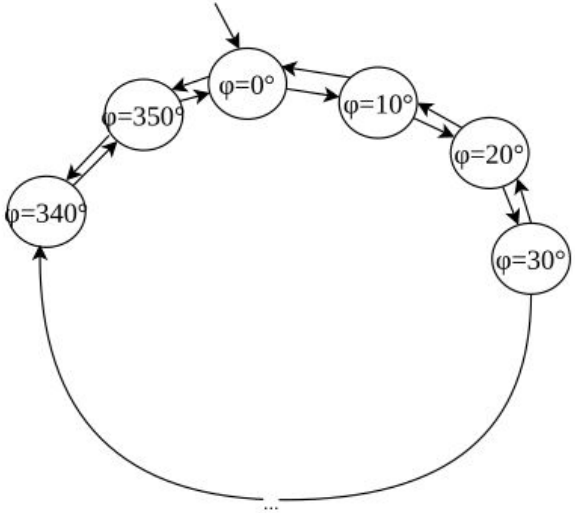
Local reachability model checking





NuSMV model of a robot

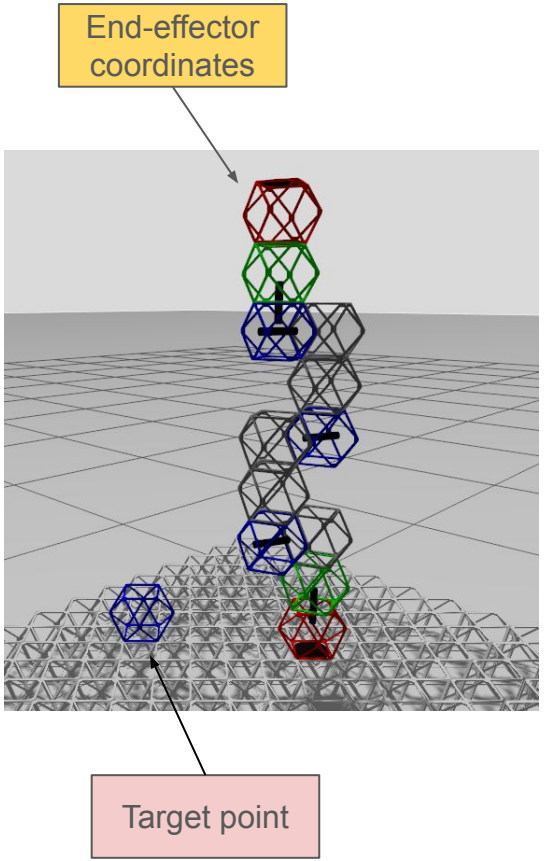
Local reachability model checking



Robot block transition system

```
MODULE block_yaw(px, py, pz,
                 xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
                 L, initYaw)
VAR
  yaw : 0..35;
  cy : cos(yaw);
  sy : sin(yaw);

DEFINE
  SCALE := 100;

  cos_yaw := cy.val;     -- [-100..100]
  sin_yaw := sy.val;

  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
  xhz2 := xhz;

  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;


ASSIGN
  init(yaw) := initYaw;
  next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

Local reachability model checking



Point reachability

```
DEFINE
  error   := 1;
  checkX := 10;
  checkY := 15;
  checkZ := 25;
  endX_inside_limits := (X_end <=(checkX+error) & X_end >=(checkX-error));
  endY_inside_limits := (Y_end <=(checkY+error) & Y_end >=(checkY-error));
  endZ_inside_limits := (Z_end <=(checkZ +error) & Z_end >=(checkZ-error));
```

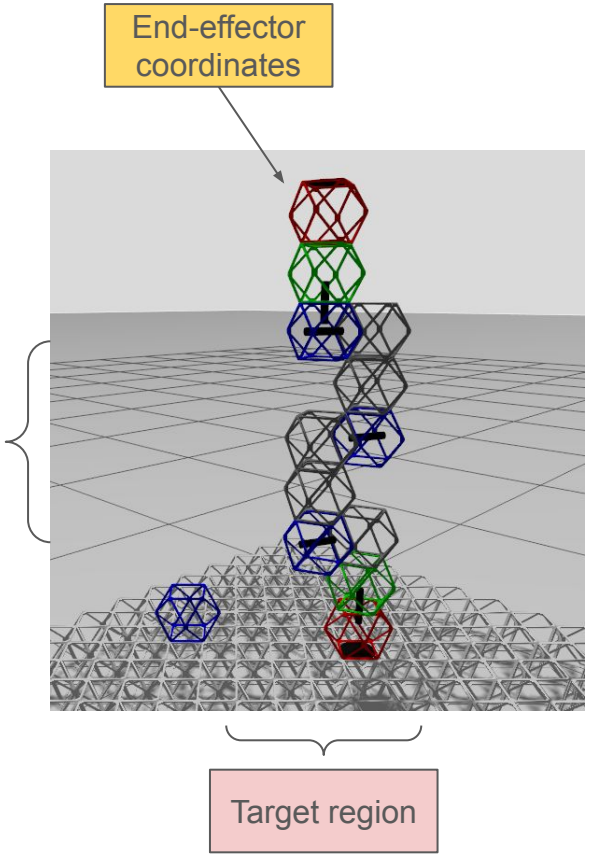
Target point

```
LTLSPEC   G !( endX_inside_limits & endY_inside_limits & endZ_inside_limits )

CTLSPEC  AG !( endX_inside_limits & endY_inside_limits & endZ_inside_limits )
```

Does not hold
Counterexample → **Point reachable**

Hold → **Point unreachable**

Local reachability model checking



Region reachability

```
FROZENVAR
   checkX : -10..10;
   checkY : -10..10;
   checkZ : 0..20;
DEFINE
  error            := 1;
  endX_inside_limits := (X_end <=(checkX+error) & X_end >=(checkX-error));
  endY_inside_limits := (Y_end <=(checkY+error) & Y_end >=(checkY-error));
  endZ_inside_limits := (Z_end <=(checkZ +error) & Z_end >=(checkZ-error));
```

Target region

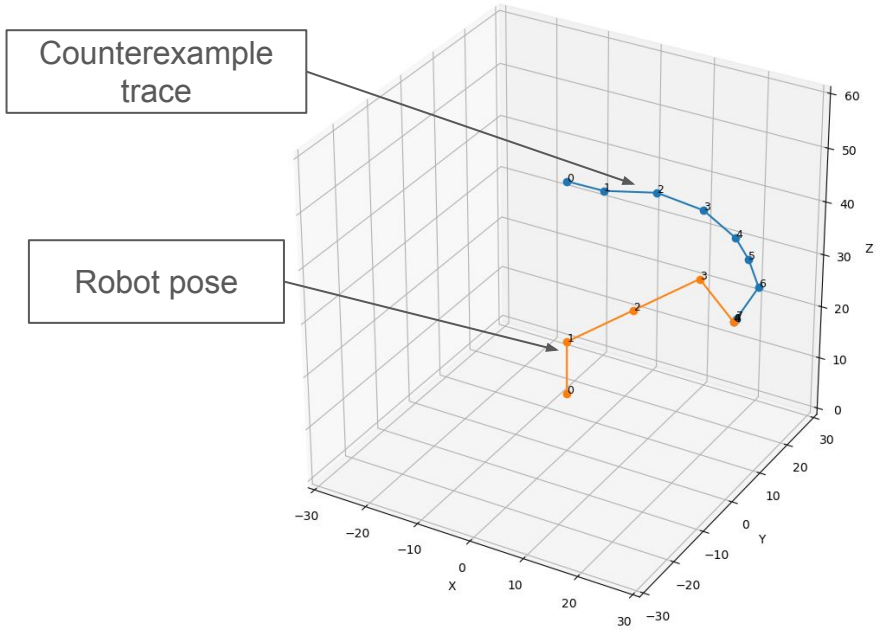
```
LTLSPEC   G !( endX_inside_limits & endY_inside_limits & endZ_inside_limits )

CTLSPEC  AG !( endX_inside_limits & endY_inside_limits & endZ_inside_limits )
```

Hold	Does not hold
All region points are reachable	**There are unreachable points**

Counterexample based control in local space

Counterexample
- plot_lines(points: list)
+ plot_trace(robot_step: int)
+ filter_variables(vars: list): list
+ get_angles(angles: list, discret : float): list
+ get_blocks(length : int): list
+ get_coordinates(var_names : list): list

Python module for automatic verification, parsing, and representation



Counterexample based control in local space

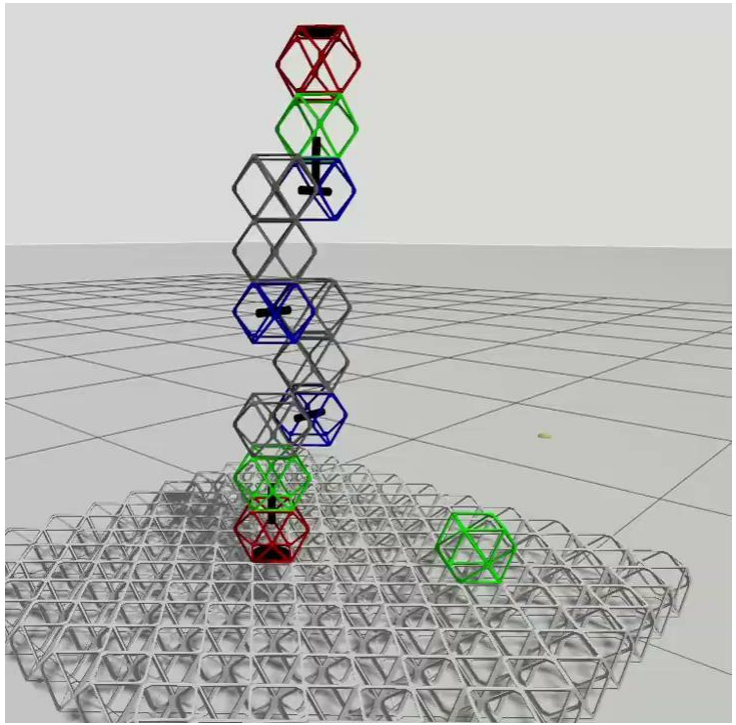
Point reachability validation through simulation

Counterexample

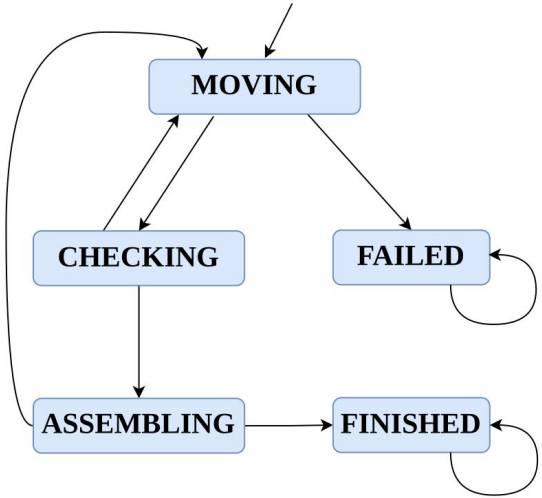
- plot_lines(points: list)
+ plot_trace(robot_step: int)
+ filter_variables(vars: list): list
+ get_angles(angles: list, discret : float): list
+ get_blocks(length : int): list
+ get_coordinates(var_names : list): list

trace:
[0, 0, 0, 0, 0]
[0, -10, -10, -10, -10]
[0, -20, -20, -20, -20]
[0, -30, -30, -30, -30]
[0, -30, -40, -40, -40]
[0, -30, -50, -50, -50]
[0, -30, -60, -50, -60]
[0, -30, -70, -50, -60]
[0, -30, -80, -50, -60]

ROS2
Control

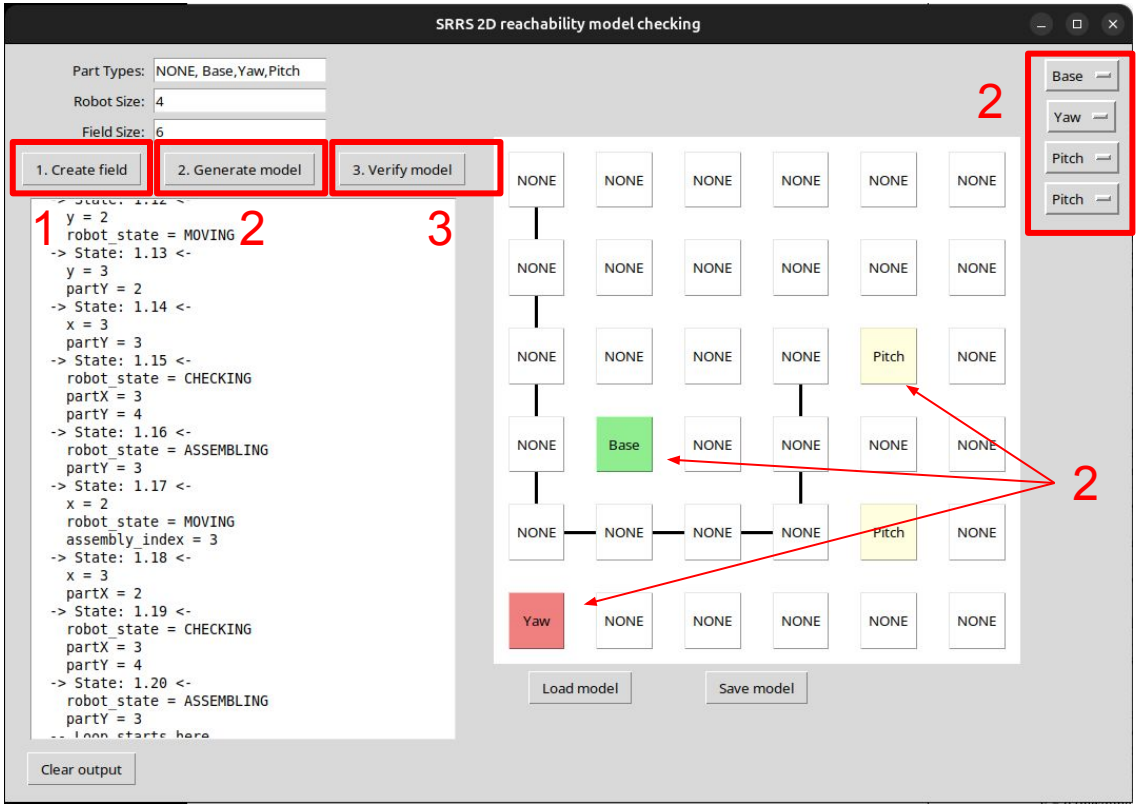


Reachability model checking in 2D space

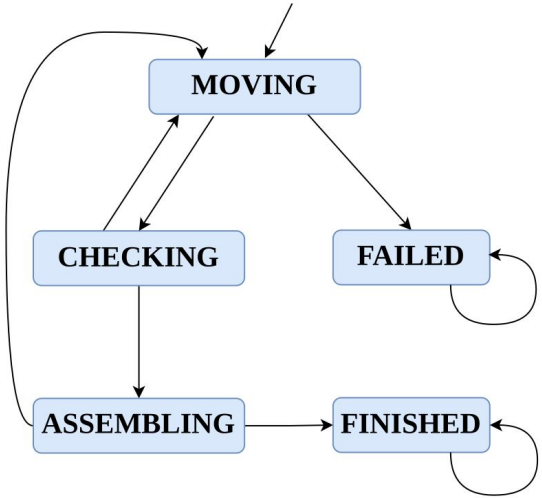


Robot transition system

GUI

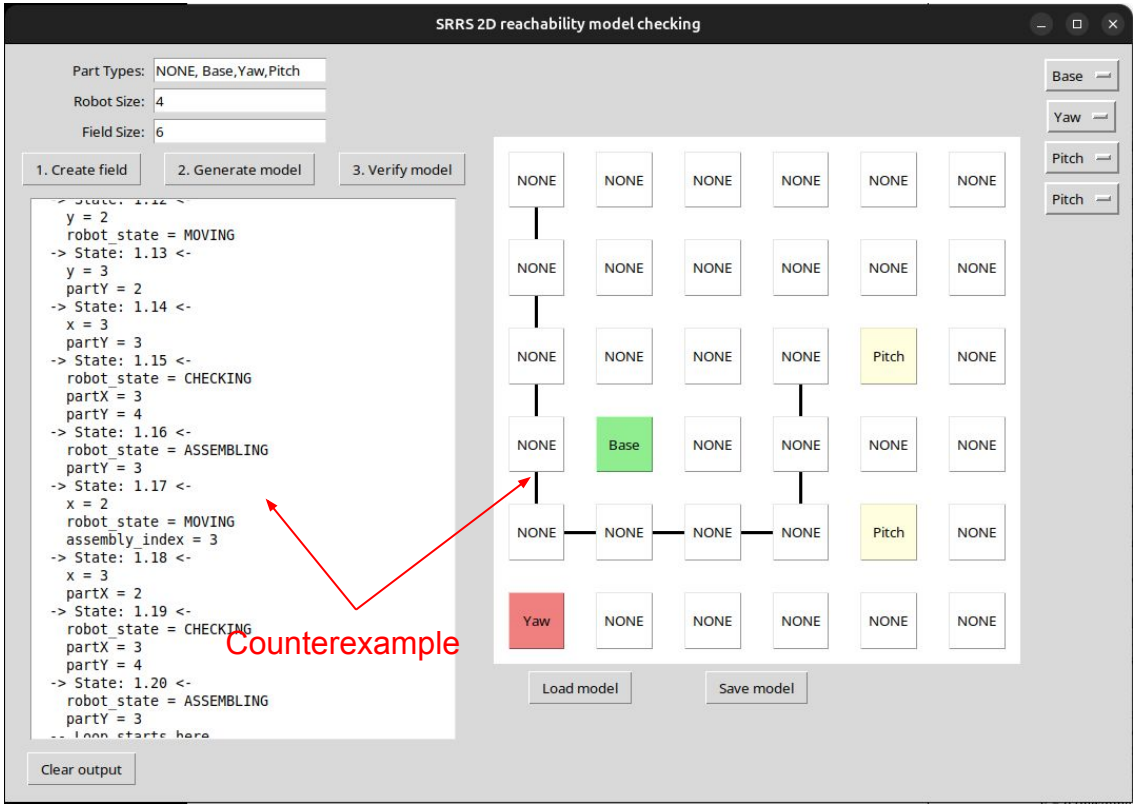


Reachability model checking in 2D space

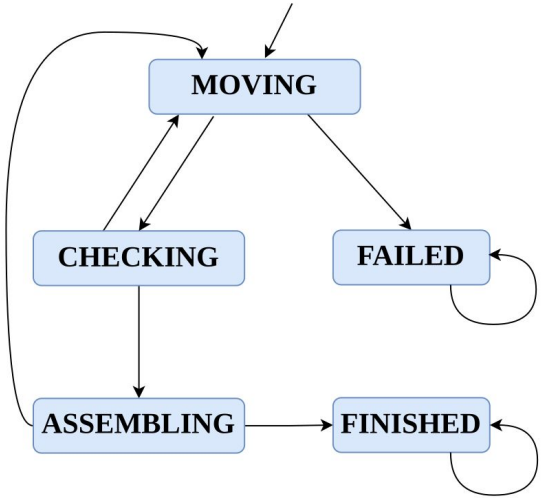


Robot transition system

GUI

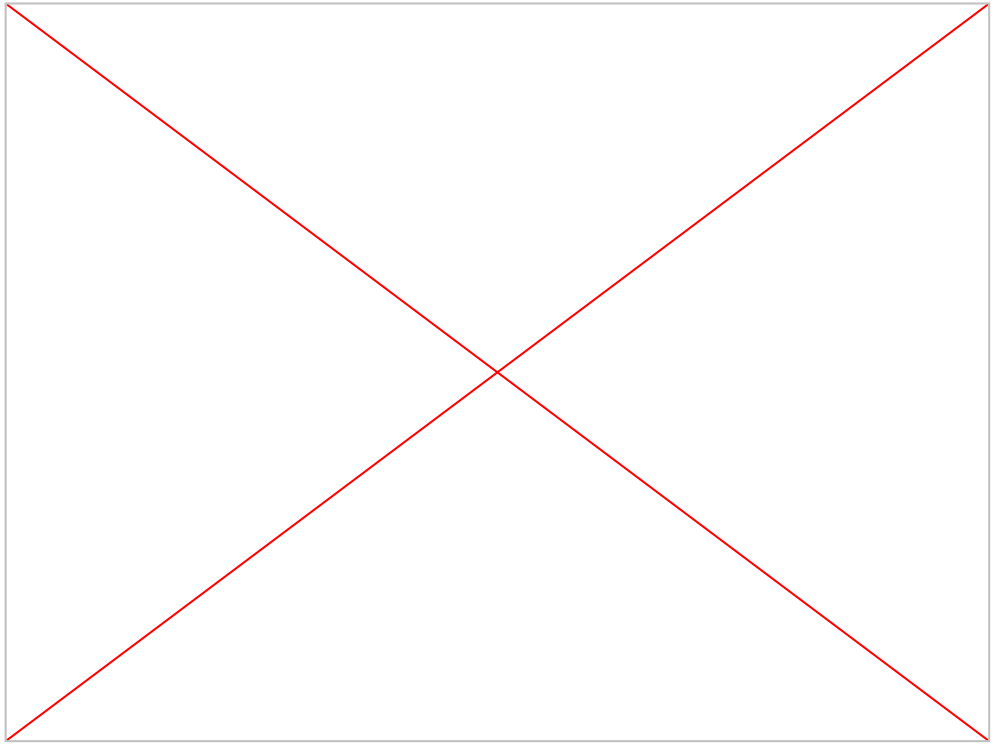


Reachability model checking in 2D space



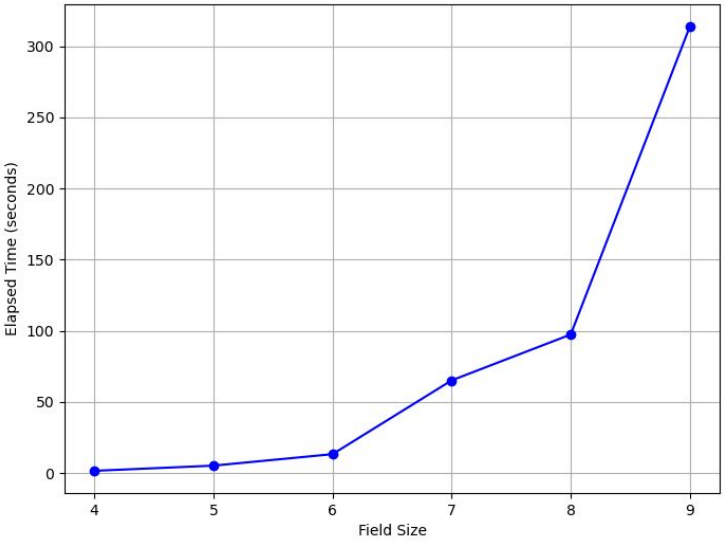
Robot transition system

Simulation

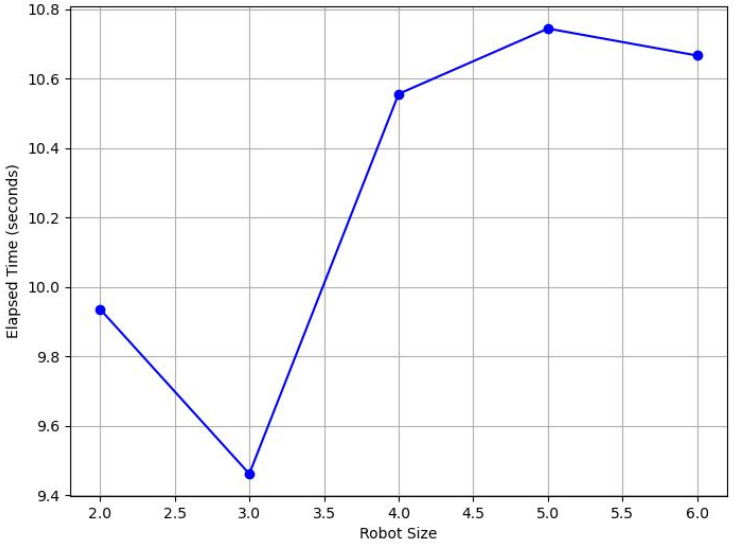


Evaluation

Computation time for checking 2D reachability



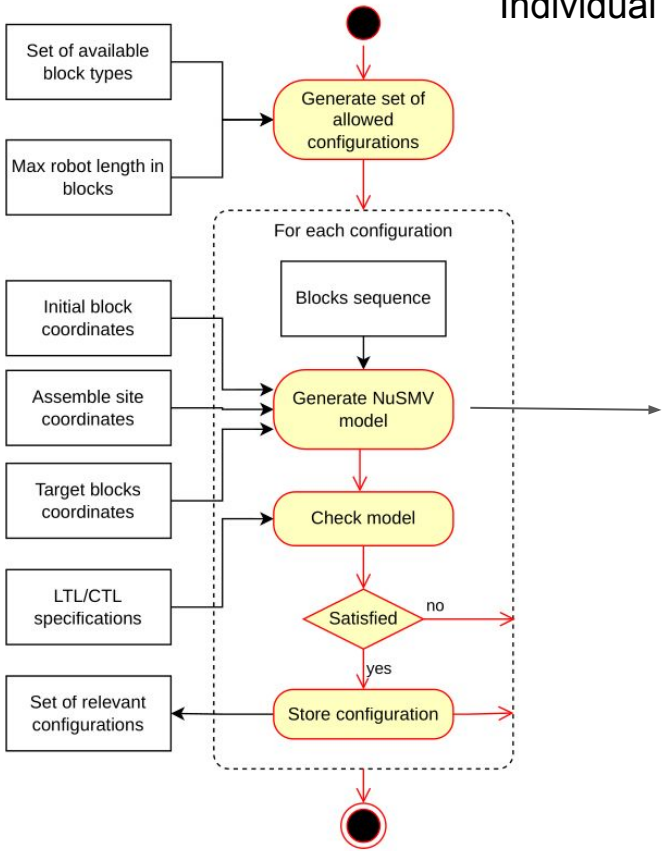
Model checking time with field size



Model checking time with robot size

Robot configuration model checking

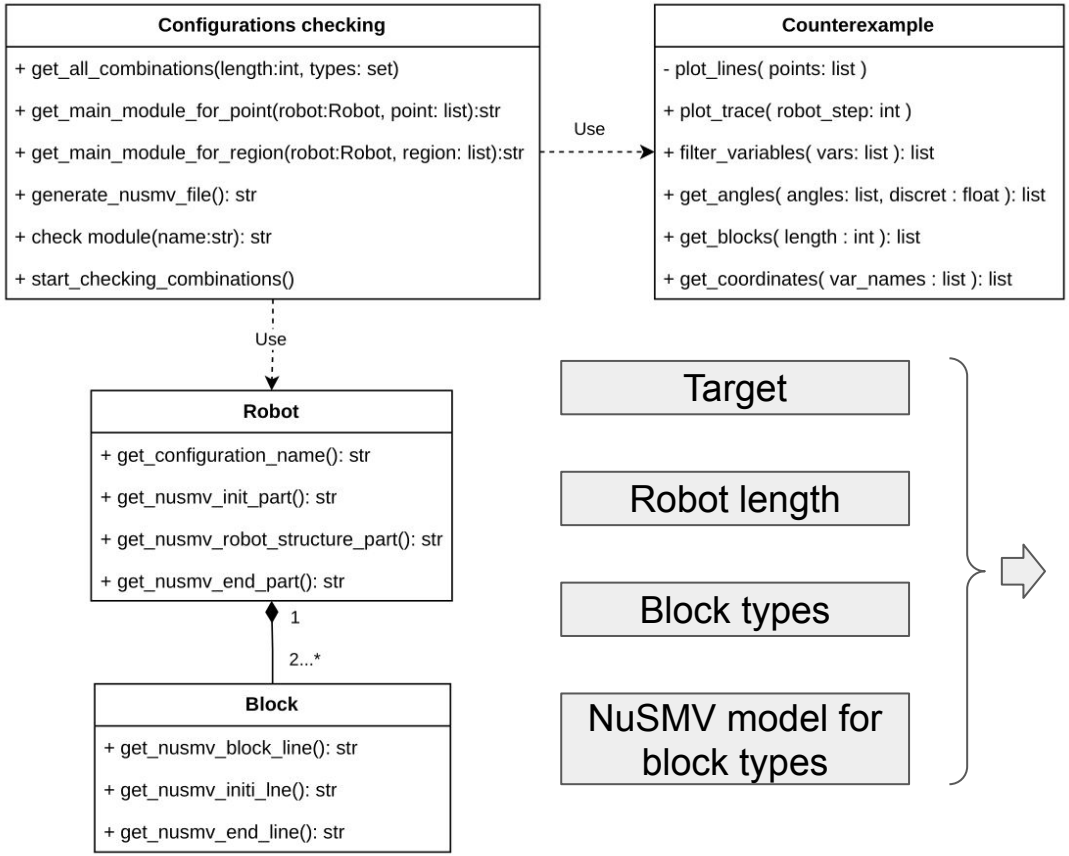
Individual model checking of robot configurations




```
MODULE main

FROZENVAR

    checkX : -10..10;
    checkY : -10..10;
    checkZ : 0..10;
VAR
    block0 : block_base(base_px, base_py, base_pz,
                        base_xhx, base_xhy, base_xhz,
                        base_yhx, base_yhy, base_yhz,
                        base_zhx, base_zhy, base_zhz,
                        L1);
    block1 : block_yaw(block0.px, block0.py, block0.pz,
                        block0.xhx, block0.xhy, block0.xhz,
                        block0.yhx, block0.yhy, block0.yhz,
                        block0.zhx, block0.zhy, block0.zhz,
                        L1, yaw1 );
    block2 : block_pitch(block1.px2, block1.py2, block1.pz2,
                        block1.xhx2, block1.xhy2, block1.xhz2,
                        block1.yhx2, block1.yhy2, block1.yhz2,
                        block1.zhx2, block1.zhy2, block1.zhz2,
                        L2, pitch2);
    block3 : block_pitch(block2.px2, block2.py2, block2.pz2,
                        block2.xhx2, block2.xhy2, block2.xhz2,
                        block2.yhx2, block2.yhy2, block2.yhz2,
                        block2.zhx2, block2.zhy2, block2.zhz2,
                        L3, pitch3);
    block4 : b
    block5 : b
```

```
    error := 1; -- +-1 (error=2) endX and endY coordinates error while checking
        reachability
    endX_inside_limits := (endX <= (checkX + error) & endX >= (checkX - error));
    endY_inside_limits := (endY <= (checkY + error) & endY >= (checkY - error));
    endZ_inside_limits := (endZ <= (checkZ + error) & endZ >= (checkZ - error));

CTLSPEC
      EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits ) --&
          blocks_above_serface)
```

Robot configuration model checking



Template base NuSMV model generation for configuration model checking

- Target
- Robot length
- Block types
- NuSMV model for block types

Reachable robot configurations

N	Configration	Point
1	base_pitch_pitch_pitch_pitch	reachable
2	base_pitch_pitch_pitch_yaw	reachable
3	base_pitch_pitch_yaw_pitch	reachable
4	base_pitch_pitch_yaw_yaw	reachable
5	base_pitch_yaw_pitch_pitch	reachable
6	base_pitch_yaw_pitch_yaw	not reachable
7	base_pitch_yaw_yaw_pitch	reachable
8	base_pitch_yaw_yaw_yaw	not reachable
9	base_yaw_pitch_pitch_pitch	reachable

Robot configuration model checking

Aggregated model checking of robot configurations

```
MODULE main
VAR
    block0 : block_base(base_px, base_py, base_pz,
                        base_xhx, base_xhy, base_xhz,
                        base_yhx, base_yhy, base_yhz,
                        base_zhx, base_zhy, base_zhz,
                        L1);
    block1 : block_yaw(block0.px, block0.py, block0.pz,
                       block0.xhx, block0.xhy, block0.xhz,
                       block0.yhx, block0.yhy, block0.yhz,
                       block0.zhx, block0.zhy, block0.zhz,
                       L1, yaw1 );
    block2 : block_pitch(block1.px2, block1.py2, block1.pz2,
                         block1.xhx2, block1.xhy2, block1.xhz2,
                         block1.yhx2, block1.yhy2, block1.yhz2,
                         block1.zhx2, block1.zhy2, block1.zhz2,
                         L2, pitch2);
    ...
DEFINE
    ...
    yaw1    := 0;
    pitch2  := 0;
    ...
```

Main module:

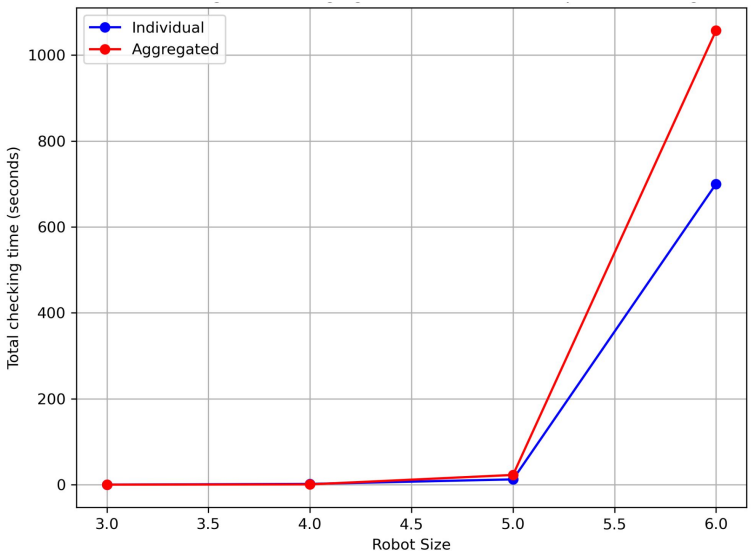
```
MODULE main
FROZENVAR
    var_block1 : {0,1};
    var_block2 : {0,1};
    ...
VAR
    block0 : block_base(base_px, base_py, base_pz,
        base_xhx, base_xhy, base_xhz,
        base_yhx, base_yhy, base_yhz,
        base_zhx, base_zhy, base_zhz, L0);
    block1 : block_general(var_block1, block0.px2,
        block0.xhx2, block0.xhy2, block0.xhz2,
        block0.yhx2, block0.yhy2, block0.yhz2,
        block0.zhx2, block0.zhy2, block0.zhz2, L1,
    block2 : block_general(var_block2, block1.px2,
        block1.xhx2, block1.xhy2, block1.xhz2,
        block1.yhx2, block1.yhy2, block1.yhz2,
        block1.zhx2, block1.zhy2, block1.zhz2, L2,
    ...
DEFINE
    ...
    angle1    := 0;
    angle2    := 0;
    ...
```

Adapter module:

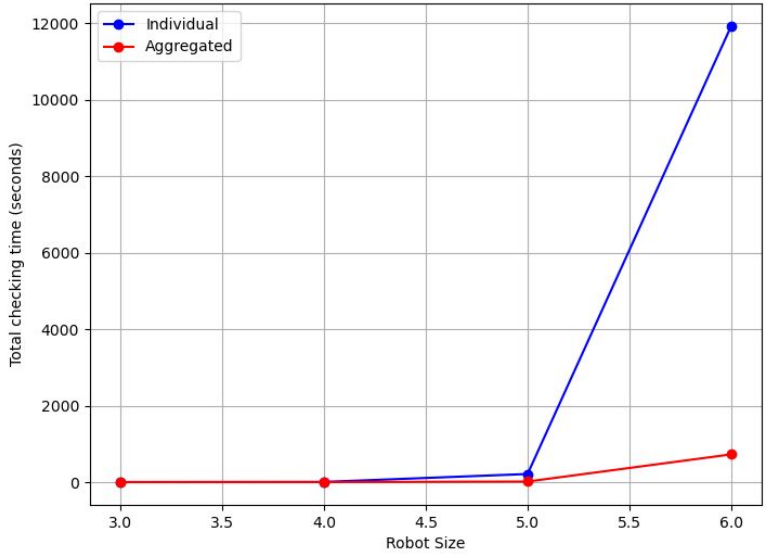
```
MODULE block_general(type, px, ..., angle)
VAR
    block_y : block_yaw  (px, ..., angle );
    block_p : block_pitch(px, ..., angle );
DEFINE
    px2    :=
        case
            type=0 : block_y.px2;
            TRUE   : block_p.px2;
        esac;
    ...
```

Robot configuration model checking

Computational time for robot configuration checking



Total time of model checking with number of blocks in robot for target point.

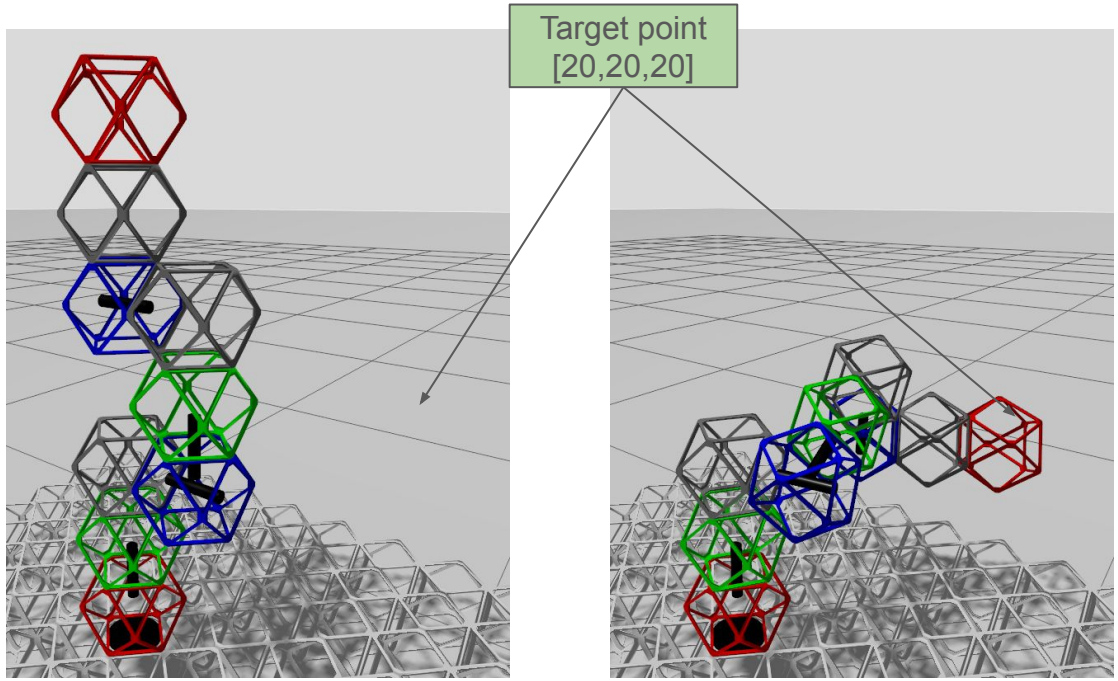


Total time of model checking with number of blocks in robot for target region.

Robot configuration model checking

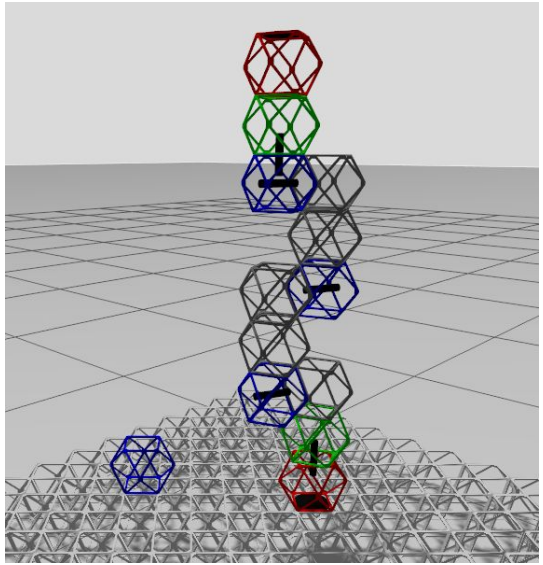
Validation through simulation

N	Configration
1	base_pitch_pitch_pitch_pitch
2	base_pitch_pitch_pitch_yaw
3	base_pitch_pitch_yaw_pitch
4	base_pitch_pitch_yaw_yaw
5	base_pitch_yaw_pitch_pitch
6	base_pitch_yaw_pitch_yaw
7	base_pitch_yaw_yaw_pitch
8	base_pitch_yaw_yaw_yaw
9	base_yaw_pitch_pitch_pitch
10	base_yaw_pitch_pitch_yaw
11	base_yaw_pitch_yaw_pitch
12	base_yaw_pitch_yaw_yaw
13	base_yaw_yaw_pitch_pitch
14	base_yaw_yaw_pitch_yaw
15	base_yaw_yaw_yaw_pitch
16	base_yaw_yaw_yaw_yaw



Threats to validity

1. **Limited generalization of the developed system:** the applicability of the proposed verification methods is confined to a specific class of SRRS systems (direct self-replication of modular robots).
2. **Discretization errors and computational trade-offs:** the accuracy and performance of the model are constrained by the discretization parameter.
3. **Inconsistencies between NuSMV models, simulation environments, and real world:** abstractions and simplifications lead to mismatches between modeled and physical behaviors.



Conclusion

1. Demonstrated how model checking can be applied to verify three aspects of SRRS behavior. Each aspect corresponds to a reachability checking on different level of system abstraction:

- Local
- Global
- Structural

2. Provided a framework for integrating formal verification in the domain of SRRS:

- Extendable NuSMV robot, model that can be used to verify different aspects of reachability.
- Python modules for NuSMV model generation for different configurations of modular robot.
- Simulation platform based on ROS2–Gazebo implementation of a modular SRRS.
- NuSMV counterexample based control

3. Described computational and modelling-related limitations of model, and how they can be mitigated

Future work :

- Model checking of other kinds of robots, material, and environments.
- Higher-level controllers capable of using counterexamples to dynamically guide assembly.
- Adaptive verification that allows the robot to modify its configuration based on environments.
- Extending the framework to collective SRRS, involving cooperate of multiple robots.